



阶段测验一解析

2400012960 祁优扬

15. x86-64 中，执行 `popq %rbx` 的操作依次为如下步骤：

- (1) 从寄存器 `%rsp` 中读出数据 `x`，以 `x` 为地址，读出对应内存中的数据 `y`；
- (2) 将 (1) 中提到的寄存器（加减多少）：加 8；
- (3) 将 `y` 存到寄存器 `%rbx` 中。

解析：`pushq` 和 `popq` 可以将数据压入程序栈中，以及从程序栈中弹出数据。栈向下增长，即栈顶元素的地址最低。栈指针 `%rsp` 保存着栈顶元素的地址。

指令	效果	描述
<code>pushq S</code>	<code>R[%rsp]←R[%rsp]-8;</code> <code>M[R[%rsp]]←S</code>	将四字压入栈
<code>popq D</code>	<code>D←M[R[%rsp]];</code> <code>R[%rsp]←R[%rsp]+8</code>	将四字弹出栈

16. 以下两组对应的 C 语言代码和汇编语言代码：

```
void multstore(long x, long y, long *dest) {  
    long t = mult2(x, y);  
    *dest = t;  
}
```

```
0000000000400640 <multstore>:  
400640: push    %rbx  
400641: mov     %rdx, %rbx  
400644: callq   400650 <mult2>  
400649: mov     %rax, (%rbx)  
40064c: pop     %rbx  
40064d: retq
```

```
long mult2(long a, long b) {
    long s = a * b;
    return s;
}
```

```
0000000000400650 <mult2>:
  400650: mov  %rdi, %rax
  400653: imul %rsi, %rax
  400657: retq
```

(1) 假设准备执行 `callq` 指令时, `%rsp` 内容为 `0x110`, 那执行完 `callq` 指令后, `%rsp` 内容为 `0x108`, `%rsp` 指向的内存中的数值为 `0x400649`。随后执行完地址 `400657` 处的 `retq` 指令后, `%rsp` 内容为 `0x110`。(均为十六进制)

解析: 在 x86-64 机器中, 指令 `call Q` 会把地址 `A` 压入栈中, 并将 `PC` 设置为 `Q` 的起始地址。压入的地址 `A` 被称为返回地址, 是紧跟在 `call` 指令后面的那条指令的地址。对应的指令 `ret` 会从栈中弹出地址 `A`, 并把 `PC` 设置为 `A`。

(2) C 代码中的下列变量分别保存在哪个寄存器中:

变量	寄存器
x	%rdi
y	%rsi
a	%rdi
b	%rsi
s	%rax

解析: `multstore` 中, `x` 为第一个参数, `y` 为第二个参数, 分别存放在 `%rdi` 和 `%rsi` 中。
`mult2` 中, `a` 为第一个参数, `b` 为第二个参数, `s` 为返回值, 分别存放在 `%rdi`、`%rsi` 和 `%rax` 中。

(3) 汇编代码中, 地址 `400641` 的指令修改了 `%rbx`, 地址 `400649` 的指令使用了 `%rbx`, 而二者中间有过程调用 `call`, 是什么机制保证了 `%rbx` 不会被修改:

答案: 根据惯例, 寄存器 `%rbx`、`%rbp` 和 `%r12 ~ %r15` 被划分为被调用者保存寄存器。当过程

P 调用过程 Q 时，Q 必须保存这些寄存器的值，保证它们的值在 Q 返回到 P 时与 Q 被调用时是一样的（要么就是根本不去改变它，要么就是把原始值压入栈中，改变寄存器的值，然后在返回前从栈中弹出旧值）。压入寄存器的值会在栈帧中创建标号为“保存的寄存器”的一部分。有了这条惯例，P 的代码就能安全地把值存在被调用者保存寄存器中（当然，要先把之前的值保存到栈上），调用 Q，然后继续使用寄存器中的值，不用担心值被破坏。

(4) 如果把 C 代码中的 `mult2(x,y)` 改为 `mult2(y,x)`，汇编代码会有什么变化：

答案：在执行 `callq` 指令之前，先交换 `%rdi` 和 `%rsi` 的值。下面这段汇编代码可以实现这一功能。

```
mov %rdi, %rax
mov %rsi, %rdi
mov %rax, %rsi
```

17. 考虑以下 C 语言代码：

```
long bit_not1(long a) {
    long x = ~a;
    return x;
}
long bit_not2(long *p) {
    long x = *p;
    long y = ~x;
    return y;
}
long call_bitnot() {
    long v0 = 1010;
    long v1 = 1010;
    long v2 = bit_not1(v0);
    long v3 = bit_not2(&v1);
    return v2 + v3;
}
```

下列变量分别存放在寄存器还是内存中？

变量	存放位置
v0	寄存器

变量	存放位置
v1	内存
v2	寄存器
v3	寄存器

解析：局部数据默认存放在寄存器中，不过有些时候，局部数据必须存放在内存中，常见的情况包括：

- 寄存器不足够存放所有的本地数据。
- 对一个局部变量使用地址运算符 `&`，因此必须能够为它产生一个地址。
- 某些局部变量是数组或结构，因此必须能够通过数组或结构引用被访问到。

由于代码中出现了 `&v1`，所以 `v1` 必须存放在内存中，而 `v0`、`v2` 和 `v3` 则存放在寄存器中。